

Digital System Design

Dr. Nazanin Ghasemian

Fall 2026 · Lecture 4



Last Lecture

- Reviewed some more combinational logic design
- Introduced Hardware Description Languages
- Talk about the design and testbench

Sequential Circuit

Electronic consumer products have the ability to

- send,
- receive,
- store,
- retrieve,
- and process information

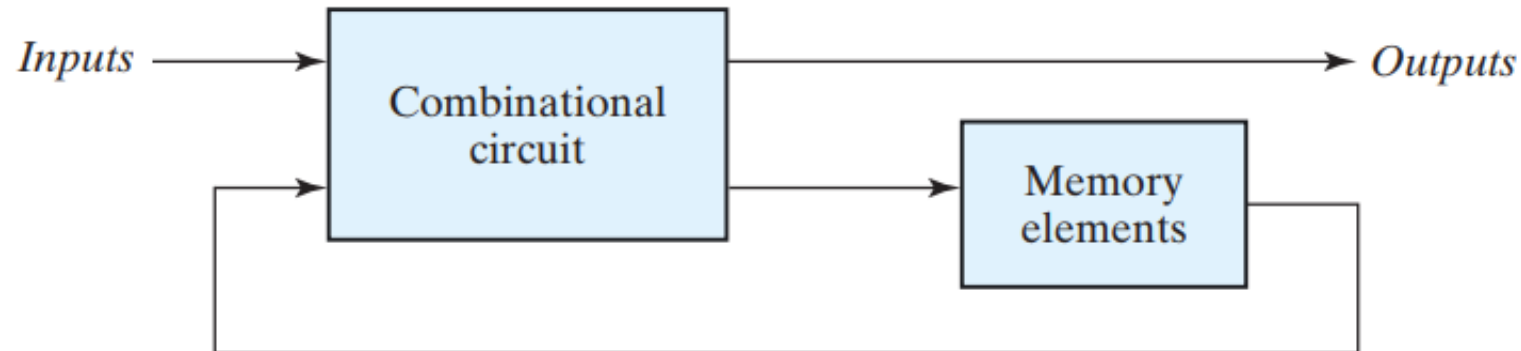
Sequential Circuit

In sequential circuits output depends on:

1. Current input
2. State of the circuit
 - Previous inputs



Memory



Synchronous / Asynchronous

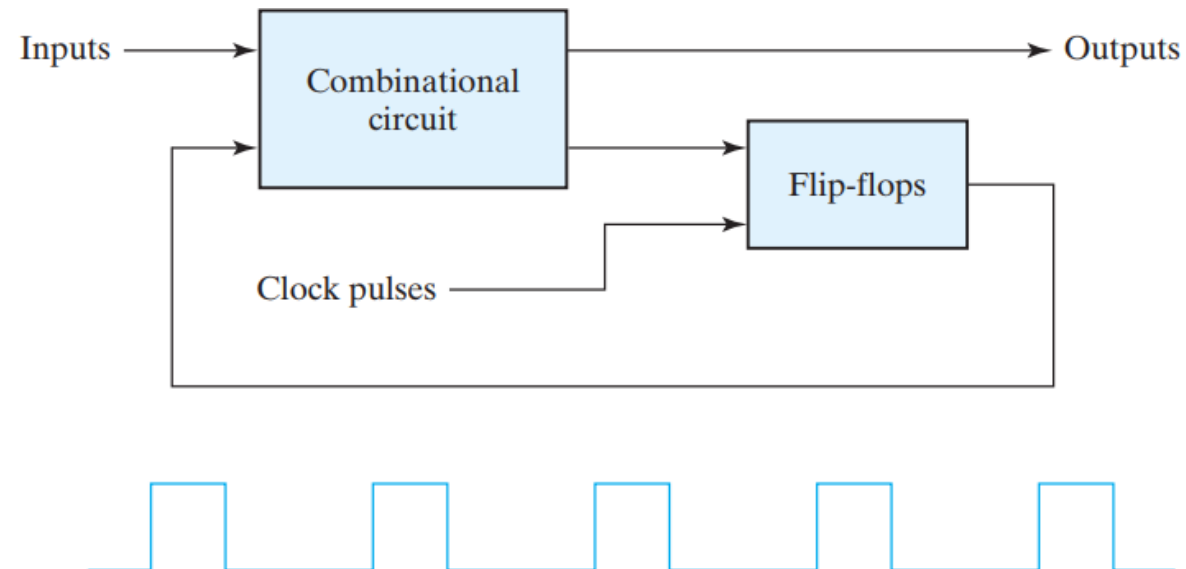
Two main types of sequential circuits:

- **Synchronous Sequential Circuits**
 - Behavior depends on signals at specific, discrete time intervals (usually controlled by a clock).
- **Asynchronous Sequential Circuits**
 - Behavior depends on input signals the order in which inputs change.
 - Also influenced by the delay of internal elements.

Synchronization - Clock

Synchronization is achieved by a timing device called a ***clock generator***, which provides ***clock pulses***.

- Clock pulses determine when computational activity will occur

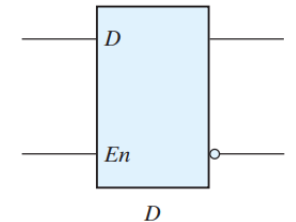
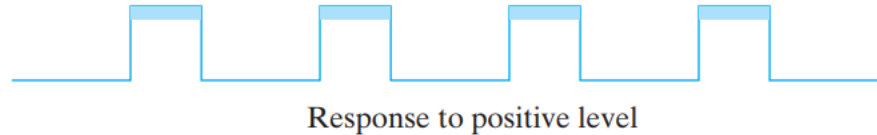


Storage Elements

A storage element can maintain a binary state as long as power is delivered to the circuit.

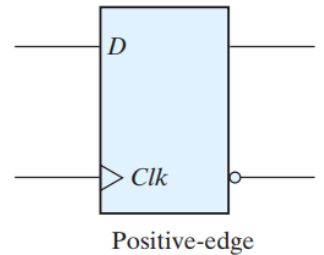
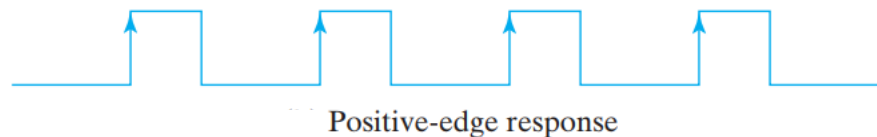
- **Latches**

- operate with signal levels (level sensitive devices)
- latches are the basic circuits from which all flip-flops are constructed.

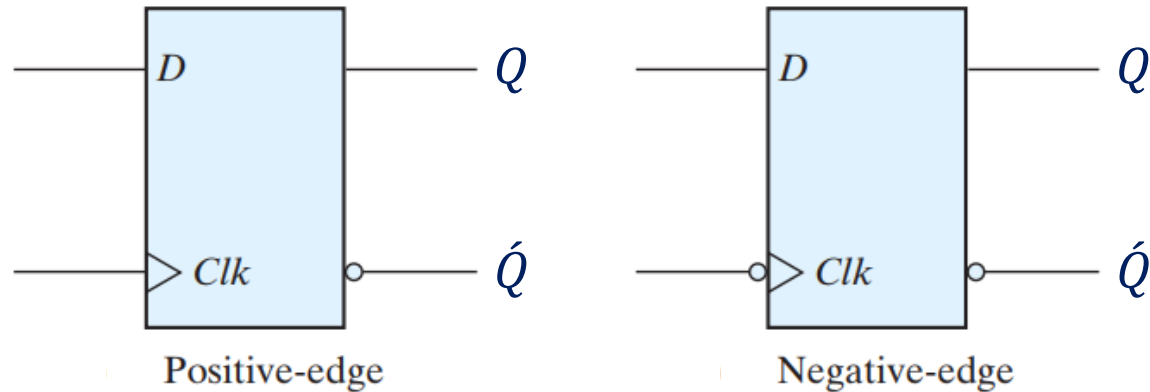


- **Flip-flops**

- those controlled by a clock transition (edge-sensitive devices)



D- Flip-Flop



Characteristic Table

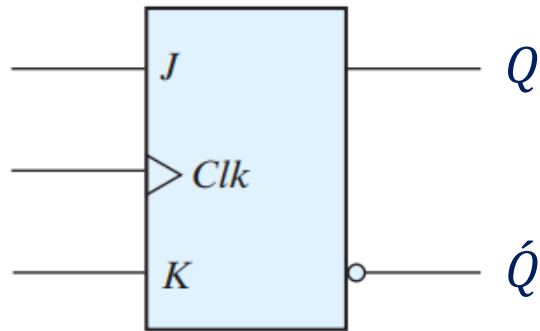
D Flip-Flop		
<i>D</i>	<i>Q(t + 1)</i>	
0	0	Reset
1	1	Set

Characteristic Equation

$$Q(t + 1) = D$$

$Q(t + 1)$, next state. The state after the clock transition.

JK- Flip-Flop



Characteristic Table

JK Flip-Flop			
<i>J</i>	<i>K</i>	$Q(t + 1)$	
0	0	$Q(t)$	No change
0	1	0	Reset
1	0	1	Set
1	1	$Q'(t)$	Complement

Characteristic

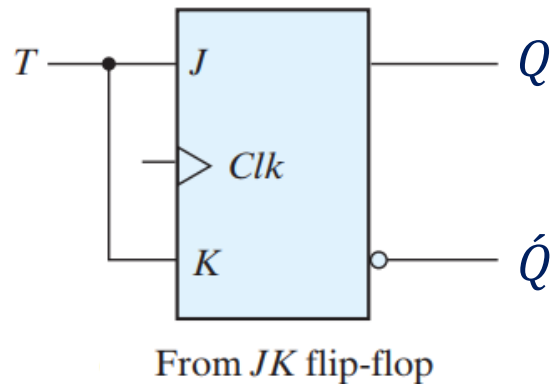
Equation

$$Q(t + 1) = JQ' + K'Q$$

$Q(t)$, present state. The state of the flip-flop immediately before the clock edge.

$Q(t + 1)$, next state. The state after the clock transition.

T- Flip-Flop



Characteristic Table

T Flip-Flop		
T	Q(t + 1)	
0	Q(t)	No change
1	Q'(t)	Complement

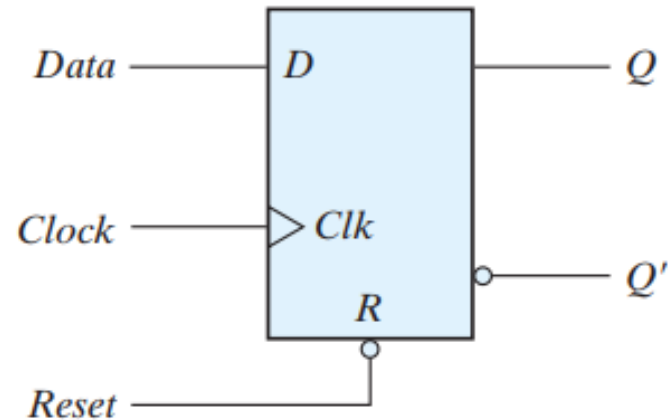
Characteristic Equation

$$Q(t + 1) = T \oplus Q = TQ' + T'Q$$

$Q(t)$, present state. The state of the flip-flop immediately before the clock edge.

$Q(t + 1)$, next state. The state after the clock transition.

D- Flip-Flop with Reset



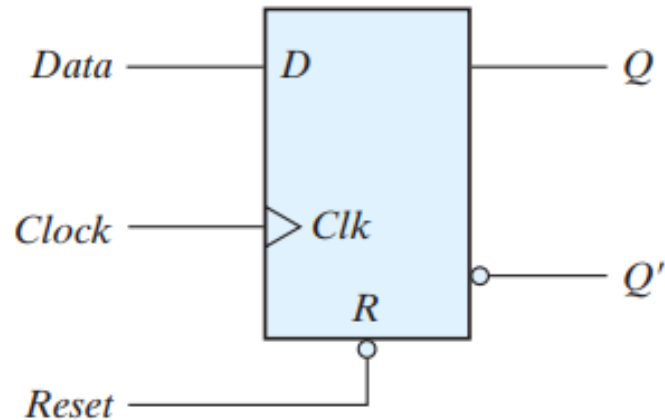
```
module D_FlipFlop(  
    input  logic D,           // Data input  
    input  logic clk,        // Clock input  
    input  logic rst_n,      // Active-low reset  
    output logic Q           // Output  
);
```

R	Clk	D	Q	Q'
0	X	X	0	1
1	↑	0	0	1
1	↑	1	1	0

The bubble on the reset input indicates that it is active-low.

- The circuit resets when $R = 0$.
- The circuit operates normally when $R = 1$.

D- Flip-Flop with Reset



➡

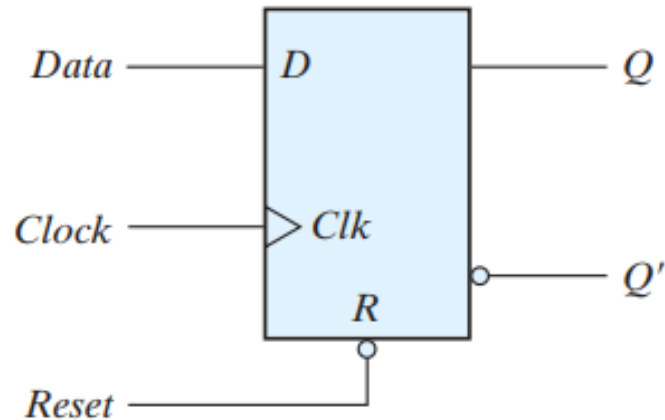
```
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        Q <= 1'b0;    // Reset output to 0
    else
        Q <= D;        // On clock edge, latch D into Q
end
```

<i>R</i>	<i>Clk</i>	<i>D</i>	<i>Q</i>	<i>Q'</i>
0	X	X	0	1
1	↑	0	0	1
1	↑	1	1	0

always_ff:

- A SystemVerilog keyword specifically used for sequential logic (flip-flops, registers).
- It make sure your code get translated to legal flip-flop code.

D- Flip-Flop with Reset



➔

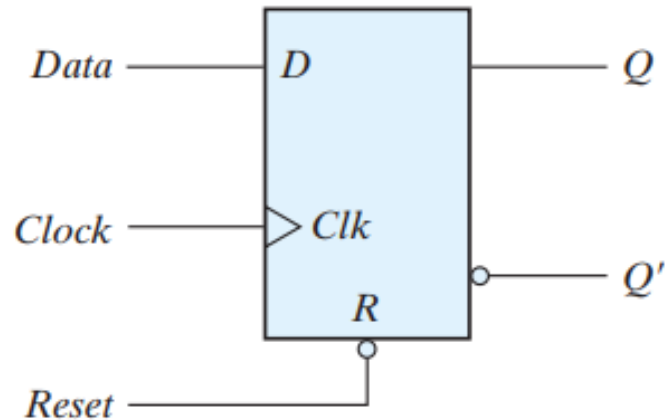
```
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        Q <= 1'b0;    // Reset output to 0
    else
        Q <= D;       // On clock edge, latch D into Q
end
```

<i>R</i>	<i>Clk</i>	<i>D</i>	<i>Q</i>	<i>Q'</i>
0	X	X	0	1
1	↑	0	0	1
1	↑	1	1	0

@(posedge clk or negedge rst_n):

- This is the **sensitivity list**. It tells the block when to execute.
- posedge clk = trigger on the rising edge of the clock.
- negedge rst_n = trigger on the falling edge of the reset signal.
- Together, this means: update either when the clock rises or when reset goes low.

D- Flip-Flop with Reset



```
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        Q <= 1'b0; // Reset output to 0
    else
        Q <= D;    // On clock edge, latch D into Q
end
```

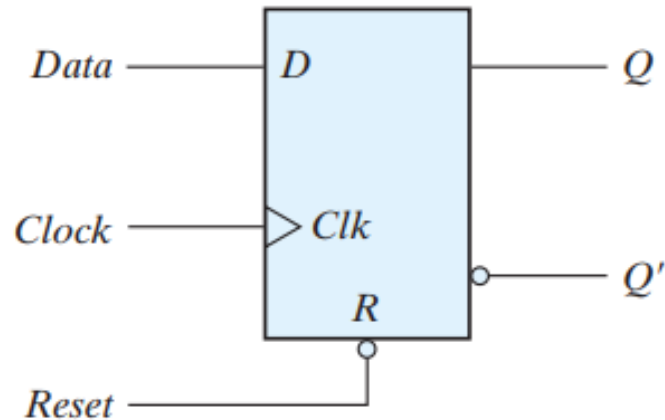
<i>R</i>	<i>Clk</i>	<i>D</i>	<i>Q</i>	<i>Q'</i>
0	X	X	0	1
1	↑	0	0	1
1	↑	1	1	0

If/else: Conditional statement.

- In SystemVerilog, an if–else statement like this can only appear inside a procedural block (e.g., always, always_ff, always_comb, initial).

!rst_n: The ! means “NOT.” (logic not)

D- Flip-Flop with Reset



```
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        ➡ Q <= 1'b0;    // Reset output to 0
    else
        Q <= D;        // On clock edge, latch D into Q
end
```

<i>R</i>	<i>Clk</i>	<i>D</i>	<i>Q</i>	<i>Q'</i>
0	X	X	0	1
1	↑	0	0	1
1	↑	1	1	0

Q <=:

- This is a non-blocking assignment (used in sequential logic).
- It means “update Q at the end of this clock cycle event.”
- This prevents race conditions when multiple flip-flops are updated in parallel.

Blocking vs Non-blocking Assignments

Blocking assignment (=)

- **Syntax:** `a = b;`
- Executes immediately, one after another, like regular programming languages (C, ...).
- Each statement **blocks** the next one until it finishes.
- **Used in:**
 - **Combinational logic** (`always_comb`, `always @*`)
 - **Initial blocks** (testbenches, stimulus code)

Blocking vs Non-blocking Assignments

Blocking assignment (=)

```
always_comb begin
    x = a & b;    // First, compute x
    y = x | c;    // Then, compute y using new x
end
```

Blocking vs Non-blocking Assignments

Non-blocking assignment (`<=`)

- **Syntax:** `a <= b;`
- Executes in **parallel** with other non-blocking assignments.
- Value is updated at the **end of the time step / clock edge**.
- Prevents race conditions in sequential logic.
- **Used in:**
 - **Sequential logic** (`always_ff`)
 - Flip-flop and registers

Blocking vs Non-blocking Assignments

Non-blocking assignment (<=)

```
always_ff @(posedge clk) begin
    q1 <= d;    // q1 gets d at clock edge
    q2 <= q1;    // q2 gets old q1, not the new one
end
```

Continuous assignment statement

assign statement

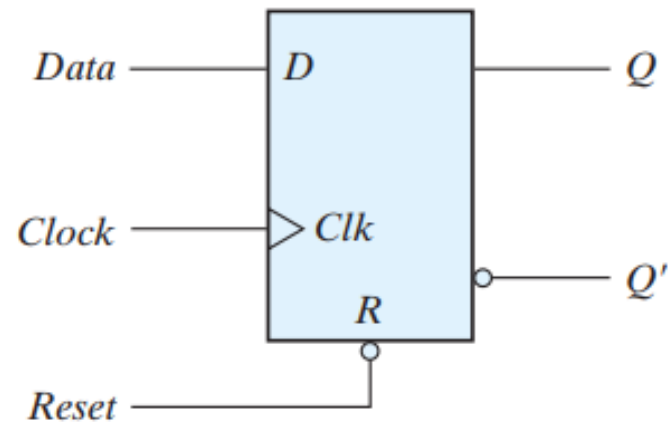
- Used outside procedural blocks (not inside `always`, `always_ff`, or `initial`).
- Continuously drives a net type (like wire).
- Anytime the right-hand expression changes, the left-hand signal is updated *immediately*.

```
assign out = a & b;    // out is always the AND of a and b
```

Assignments

- `=` (blocking) → inside procedural blocks, step-by-step.
- `<=` (non-blocking) → inside procedural blocks, parallel updates at clock edges.
- **assign** → outside procedural blocks, continuous connection (like a physical wire).

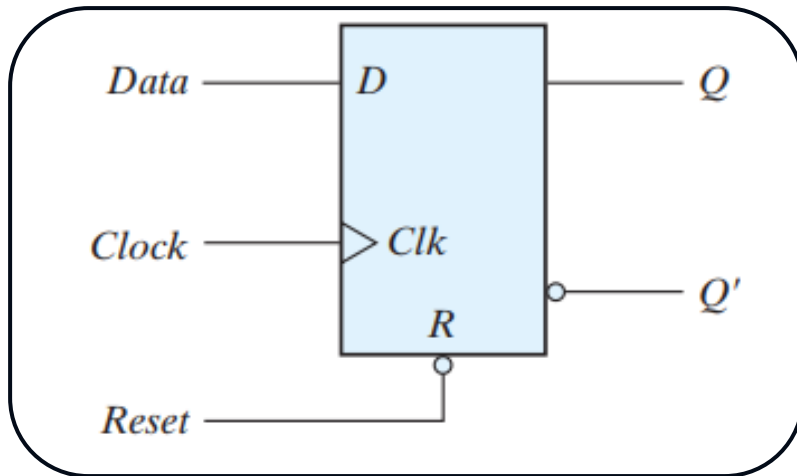
D- Flip-Flop with Reset



R	Clk	D	Q	Q'
0	X	X	0	1
1	↑	0	0	1
1	↑	1	1	0

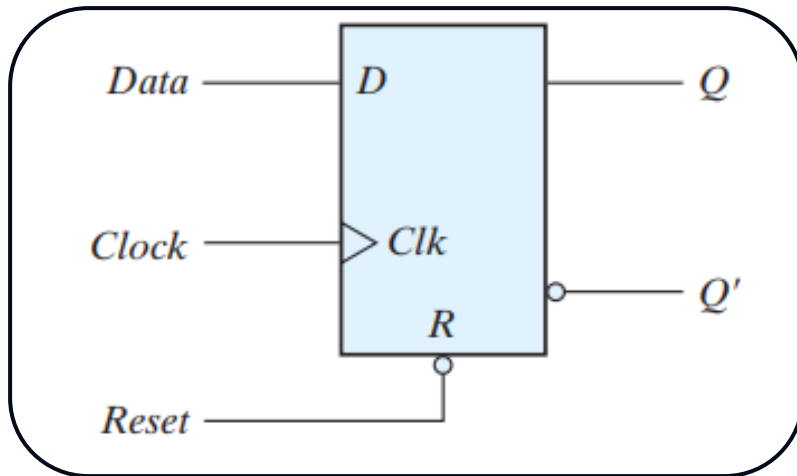
```
module D_FlipFlop(  
    input  logic D,          // Data input  
    input  logic clk,        // Clock input  
    input  logic rst_n,      // Active-low reset  
    output logic Q           // Output  
);  
  
    always_ff @(posedge clk or negedge rst_n) begin  
        if (!rst_n)  
            Q <= 1'b0;      // Reset output to 0  
        else  
            Q <= D;         // On clock edge, latch D into Q  
        end  
    endmodule
```

D- Flip-Flop with Reset - Testbench

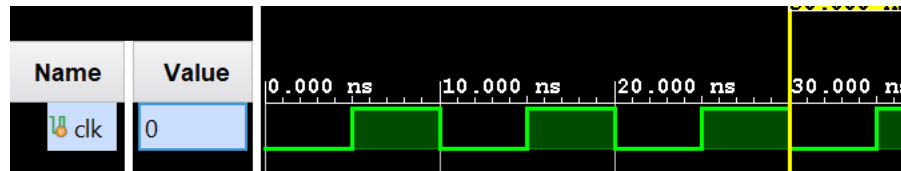


```
module tb_D_FlipFlop;  
  
    // Testbench signals  
    logic D;  
    logic clk;  
    logic rst_n;  
    logic Q;  
  
    // Instantiate the D Flip-Flop  
    D_FlipFlop uut (  
        .D(D),  
        .clk(clk),  
        .rst_n(rst_n),  
        .Q(Q)  
    );  
endmodule
```

D- Flip-Flop with Reset - Testbench



```
// Clock generation: 10ns period  
always #5 clk = ~clk;
```

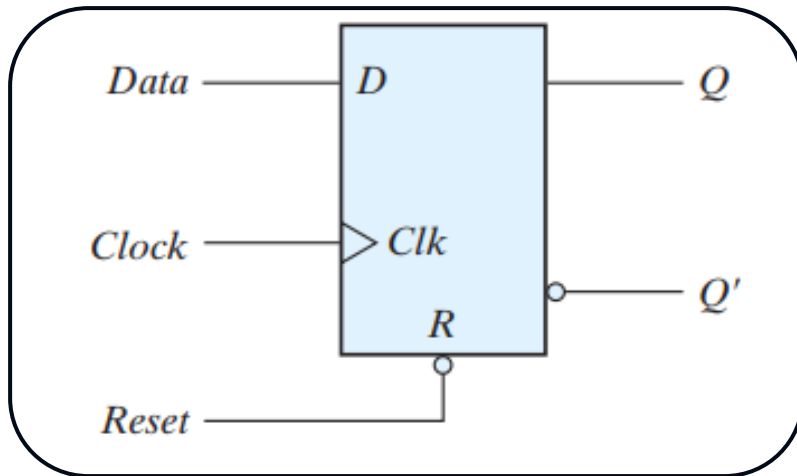


Clock period = 10 ns (10 time units)

- **Always** : Starts a procedural block that repeats forever during simulation. No begin/end here, since it's just one statement.
- **#5** “wait 5 time units”
- **clk = ~clk;** Assigns the inverted value back to clk.

Effect: toggles the clock signal each time it runs.

D- Flip-Flop with Reset - Testbench



```
initial begin
    // Initialize signals
    clk    = 0;
    rst_n  = 0;
    D      = 0;

    // Apply reset
    #10;
    rst_n = 1;    // Release reset

    // Apply test sequence
    #10 D = 1;
    #10 D = 0;
    #10 D = 1;
    #10 D = 1;
    #10 D = 0;

    // Finish simulation
    #20 $finish;
end
```